

Indian Institute of Technology, Ropar

12 August, 2026



EE208 Control Engineering Lab Report

Experiment 11

Group no.: 11A

(Saurav - 2024EPB1279

Ashwin - 2024EPB1262

Sumitha S - 2022EPB1251)

Experiment 11

State Feedback Controller Design on MATLAB Platform

1. Objective:

To design state feedback gain matrices for a given third-order discrete-time state-space system such that:

1. The maximum magnitude of the closed-loop eigenvalues is equal to the maximum magnitude specified by the nominal eigenvalue set.
2. The closed-loop system remains controllable and observable.
3. Different combinations of feedback gains are investigated to identify valid controller configurations.
4. The effect of variations in the feedback gains on closed-loop stability is studied.

2. Software / Tools Required:

Tool	Purpose
MATLAB	State-space modelling and controller design
Control System Toolbox	<code>ss</code> , <code>place</code> , <code>ctrb</code> , <code>obsv</code> , etc.
Symbolic Math Toolbox	Eigenvalue/eigenvector calculations

3. Theory / Background:

3.1 Discrete-Time State-Space Representation

A discrete-time state-space system can be represented as

$$\begin{aligned}x[k + 1] &= Ax[k] + Bu[k] \\ y[k] &= Cx[k] + Du[k]\end{aligned}$$

where:

- $x[k]$ is the state vector,
- $u[k]$ is the input vector,
- $y[k]$ is the output vector,
- A is the state matrix,
- B is the input matrix,
- C is the output matrix, and
- D is the direct transmission matrix.

The system considered in this experiment is a third-order system with two inputs and two outputs.

3.2 State Feedback Control

In state feedback control, the control input is determined from the measured state variables.

The control law is given by

$$u[k] = -Kx[k]$$

where K is the state feedback gain matrix.

Substituting the control law into the state equation,

$$x[k + 1] = Ax[k] + B(-Kx[k])$$

Therefore,

$$x[k + 1] = (A - BK)x[k]$$

Hence, the closed-loop state matrix is

$$A_{cl} = A - BK$$

The eigenvalues of A_{cl} determine the dynamics and stability of the closed-loop system.

3.3 Pole Placement

The state feedback gain K can be selected to place the closed-loop poles at desired locations.

The desired condition is

$$eig(A - BK) = \{p1, p2, ..., pn\}$$

MATLAB provides the `place()` function for calculating the state feedback gain:

```
K = place(A,B,p);
```

where `p` contains the desired closed-loop poles.

3.4 Stability of a Discrete-Time System

For a discrete-time system, stability requires all closed-loop eigenvalues to lie inside the unit circle in the complex plane.

Thus,

$$|\lambda_i| < 1$$

for every closed-loop eigenvalue.

The maximum eigenvalue magnitude is therefore

$$\rho = \max |\lambda_i|$$

and the system is stable when

$$\rho < 1$$

3.5 Controllability

A system is completely controllable if every state can be influenced by the system inputs.

For a third-order system, the controllability matrix is

$$C = [B \quad AB \quad A^2B]$$

The system is completely controllable if

$$rank(C) = 3$$

In MATLAB:

```
Ctrb = ctrb(A,B);
```

```
rank_ctrb = rank(Ctrb);
```

Controllability is important for state feedback because arbitrary closed-loop pole placement requires the system to be controllable.

3.6 Observability

Observability determines whether the internal states of a system can be determined from its outputs.

For a third-order system,

$$O = \begin{bmatrix} C \\ CA \\ CA^2 \end{bmatrix}$$

The system is completely observable if

$$\text{rank}(O) = 3$$

In MATLAB:

```
Obsv = obsv(A,C);  
rank_obsv = rank(Obsv);
```

3.7 Sensitivity of State Feedback Gains

The robustness of the state feedback controller can be investigated by varying individual elements of the gain matrix.

For

$$K = \begin{bmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \end{bmatrix}$$

each gain element can be varied independently while the remaining elements are kept at their nominal values.

The closed-loop matrix is recalculated as

$$Acl = A - BK$$

and its maximum eigenvalue magnitude is evaluated.

For the discrete-time system:

$$\max|\lambda| < 1$$

indicates stability.

The sensitivity analysis therefore determines the range of gain variations for which the controller continues to maintain closed-loop stability.

4. System Under Study

The plant used throughout (Parts A–D) is the discrete-time ($T_s = 1$) state-space model:

A =

1.1	0.3	-1.5
0.1	3.5	2.2
0.4	2.4	-1.1

B =

0.1	0
0	1.1
1	1

C =

1	2	0
0	0	1

D =

1	2
0	1

States $n = 3$, inputs $m = 2$, outputs $p = 2$. The nominal target closed-loop pole set specified for the exercise is $\{0.5, 0.3, -0.2\}$, i.e. a required maximum eigenvalue magnitude of 0.5.

3. Part A — Nominal State-Feedback Design

3.1 MATLAB code

```

% System definition
A = [1.1, 0.3, -1.5;
      0.1, 3.5, 2.2;
      0.4, 2.4, -1.1];
B = [0.1, 0;
      0, 1.1;
      1, 1];
C = [1, 2, 0;
      0, 0, 1];
D = [1, 2;
      0, 1];
Ts = 1;
p_nominal = [0.5, 0.3, -0.2];

n = size(A,1); m = size(B,2); p = size(C,1);

% Open-loop eigenvalues
eig_ol = eig(A);

% Controllability and observability (Kalman rank test)
Ctrb = ctrb(A,B); rank_ctrb = rank(Ctrb);
Obsv = obsv(A,C); rank_obsv = rank(Obsv);

% Guard clause: abort if the plant cannot be assigned arbitrary poles
if rank_ctrb < n
    error('System is not controllable. Aborting.');
```

end

```

if rank_obsv < n
    error('System is not observable. Aborting.');
```

end

```

% Design nominal state-feedback gain K by pole placement
K      = place(A, B, p_nominal);
A_cl = A - B*K;

% Closed-loop eigenvalues and the required performance check
eig_cl = eig(A_cl);
max_mag_nom = max(abs(p_nominal));
```

```

max_mag_cl    = max(abs(eig_cl));

% Re-verify controllability / observability AFTER feedback
Ctrb_cl = ctrb(A_cl, B); rank_ctrb_cl = rank(Ctrb_cl);
Obsv_cl = obsv(A_cl, C); rank_obsv_cl = rank(Obsv_cl);

% Initial-condition (free) response, open loop vs closed loop
x0 = [1; 0; 0];
t = 0:1:15;

sys_ol = ss(A,B,C,D,Ts);
sys_cl = ss(A_cl, zeros(size(B)), C, zeros(size(D)), Ts);

[y_ol, t_out, x_ol] = initial(sys_ol, x0, t);
[y_cl, ~, x_cl] = initial(sys_cl, x0, t);

```

3.2 What this code is doing

- `eig(A)` obtains the open-loop pole locations, used purely as a diagnostic to see how far the uncontrolled plant is from stability before any feedback is designed.
- `ctrb(A,B)` / `obsv(A,C)` followed by `rank(·)` implement the Kalman rank test. Full rank (= n) on both is the necessary and sufficient condition for an arbitrary set of n closed-loop poles to be assignable by state feedback; the two if-blocks turn this into a hard guard that aborts the script rather than letting `place()` fail on, or silently mis-solve, a plant that cannot support the requested design.
- `K = place(A,B,p_nominal)` solves for the unique matrix K such that $\text{eig}(A-BK) = p_nominal$, using the robust (Kautsky–Nichols–Van Dooren) pole-placement algorithm. Because B has $m = 2$ columns, this problem is under-determined — `place()` internally selects one particular, well-conditioned member of an entire family of gain matrices that all achieve the same three poles (this non-uniqueness is the subject of Part D).
- `A_cl = A - B*K` forms the closed-loop state matrix under the feedback law $u = -Kx$, and `eig(A_cl)` reads back the poles actually achieved, so the design can be checked against the specification (`max_mag_cl` vs `max_mag_nom`).
- `ctrb(A_cl,B)` / `obsv(A_cl,C)` repeat the rank test on the closed loop. This is not redundant with the open-loop check: state feedback changes A but not B or C , and for special

(non-generic) choices of K it is possible to introduce an unintended pole–zero cancellation that destroys minimality, so the closed-loop pair is re-verified rather than assumed safe.

- `ss(...)` and `initial(...)` build discrete state-space objects for the open- and closed-loop systems and compute their zero-input (free) response to the same initial state $x_0 = [1; 0; 0]$ over 16 samples — this is the time-domain evidence used to contrast the diverging open-loop trajectory with the poled-in, decaying closed-loop trajectory.

3.3 Results

Quantity	Value
Open-loop eigenvalues	−1.886732, 0.958171, 4.428561
Open-loop max $ \lambda $	4.428561 → plant is open-loop UNSTABLE
<code>rank(ctrb(A,B))</code>	3 (= n) → controllable
<code>rank(observ(A,C))</code>	3 (= n) → observable
Guard clauses	both passed — design proceeds

With the controllability/observability guard satisfied, `place()` was run for the nominal pole set $\{0.5, 0.3, -0.2\}$:

Quantity	Value
Nominal poles requested	0.5, 0.3, −0.2 (max $\lambda = 0.5$)
K (row 1)	0.195155 −0.484232 −1.826888
K (row 2)	−15.859694 −5.224297 10.454099
Achieved closed-loop eigenvalues	0.500, 0.300, −0.200 ($\approx$ nominal set)
Achieved max $ \lambda $	0.500 — EXACT match to specification
<code>rank(ctrb(A_cl,B))</code>	3 (= n) → still controllable
<code>rank(observ(A_cl,C))</code>	3 (= n) → still observable

The single feedback matrix K simultaneously relocates all three poles — including the two that were open-loop unstable (magnitudes 1.887 and 4.429) — to the prescribed locations, while both rank tests remain full after feedback. Because $m = 2 > 1$, this K is only one of infinitely many gain matrices that achieve exactly this pole set; Part D quantifies that redundancy directly.

4. Part B — Alternate Pole Sets at the Same Performance Level

The exercise asks whether the “max-magnitude = 0.5” specification can be met by more than one pole configuration, and how the resulting gains compare.

4.1 MATLAB code

```
% Additional pole sets, all sharing the same max magnitude = 0.5
alt_pole_sets = {
    'Set1', [0.5, 0.3, 0.2];
    'Set2', [0.5, -0.3, 0.2];
    'Set3', [0.4+0.3i, 0.4-0.3i, -0.2];
};

num_sets      = size(alt_pole_sets,1);
alt_results = cell(num_sets, 1);

for i = 1:num_sets
    pname = alt_pole_sets{i,1};
    poles = alt_pole_sets{i,2};

    Ki = place(A,B,poles);
    Acli = A - B*Ki;
    eigs_i = eig(Acli);
    maxmag_i = max(abs(eigs_i));
    rank_ctrb_i = rank(ctrb(Acli,B));
    rank_obsv_i = rank(observ(Acli,C));

    alt_results{i} = struct('Name',pname,'Poles',poles,'K',Ki, ...
        'Eig',eigs_i,'MaxMag',maxmag_i, ...
        'RankCtrb',rank_ctrb_i,'RankObsv',rank_obsv_i);
```

```
end
```

4.2 What this code is doing

Three additional pole sets are chosen, each with the same required $\max |\lambda| = 0.5$ as the nominal design, but placed differently within the disc: Set 1 uses three positive real poles, Set 2 mixes a negative real pole in, and Set 3 replaces two of the real poles with a complex-conjugate pair ($0.4 \pm 0.3j$) so that the closed loop is oscillatory rather than purely exponential. The loop repeats exactly the Part-A design/verify pattern — `place()` for the gain, $A - BK$ for the closed-loop matrix, `eig()` to confirm the poles landed correctly, and `ctrb/obsv` rank checks — for each set in turn, and stores the results for comparison. The purpose is to show that a single scalar spec (“max pole magnitude”) under-determines the transient shape: all three designs share the same asymptotic decay envelope (0.5^k), but Set 3's complex pair will produce a decaying oscillation that Sets 1–2 do not.

4.3 Results

MATLAB was not available in the report-generation environment, so the table below was produced with an equivalent robust pole-placement routine (SciPy's `place_poles`, from the same Kautsky–Nichols–Van Dooren algorithm family as MATLAB's `place`). Because $m = 2$ makes the gain non-unique, the specific numbers in K may not match a MATLAB run entry-for-entry — but the achieved eigenvalues and rank outcomes shown are exact, deterministic consequences of the design method and would hold for any valid solution.

Set	Poles requested	K (row 1 / row 2)	Closed-loop eig.	$\max \lambda $	rank ctrb / obsv
Set 1	0.5, 0.3, 0.2	−0.188, −0.744, −2.621 / 0.147, 2.936, 1.910	0.500, 0.200, 0.300	0.500	3 / 3
Set 2	0.5, −0.3, 0.2	−0.487, −0.982, −2.029 / 0.198, 3.054, 1.818	−0.300, 0.500, 0.200	0.500	3 / 3
Set 3	$0.4 \pm 0.3j$, −0.2	−0.333, −1.136, −2.220 / 0.098, 3.210, 1.623	$0.4 \pm 0.3j$, −0.200	0.500	3 / 3

All three alternate designs land exactly on their specified poles, reproduce the required $\max |\lambda| = 0.5$, and keep the closed loop fully controllable and observable — confirming that the plant's

controllability/observability margin is not fragile to which particular pole configuration is chosen, only to how large the pole magnitudes are allowed to be.

5. Part C — Gain-Sensitivity (Robustness) Analysis

This section goes beyond the base exercise to ask a practical question: if any single entry of the designed K drifts from its nominal value (actuator tolerance, quantisation, ageing, mis-implementation), how far can it move before the closed loop becomes unstable?

5.1 MATLAB code (core sweep)

```
% Plant matrices A, B, C as in Part A
K_nominal = [ 0.195155, -0.484232, -1.826888;
             -15.859694, -5.224297, 10.454099]; % nominal gain from Part A

elem_names = {'K11','K12','K13','K21','K22','K23'};
num_steps = 200; % ~1% resolution per sweep step

for idx = 1:6
    if idx <= 3
        i = 1; j = idx;
    else
        i = 2; j = idx - 3;
    end
    nom = K_nominal(i,j);

    % Sweep this ONE gain element from -100% to +100% of its nominal
    % value (i.e. from 0 up to 2*nominal), holding the other five fixed
    pct_range = linspace(-100, 100, num_steps);

    for s = 1:num_steps
        pct = pct_range(s);
        kval = nom * (1 + pct/100);

        K = K_nominal;
        K(i,j) = kval; % perturb only element (i,j)
```

```

Acl = A - B*K;
ev = eig(Acl);

maxmag = max(abs(ev));
isStable = all(abs(ev) < 1); % discrete-time (Schur) stability test
end
end

```

5.2 What this code is doing

- The loop over `idx = 1:6` walks through the six independent entries of the 2×3 gain matrix one at a time (`k11`, `k12`, `k13`, `k21`, `k22`, `k23`), using the index arithmetic `i,j` to address the right (row, column) pair inside `K_nominal`.
- For each element, `pct_range` linearly sweeps the percentage change from -100% to $+100\%$ of that element's nominal value, i.e. the element itself is driven from 0 up to twice its nominal value, in 200 equal steps ($\approx 1\%$ resolution per step).
- Inside the inner loop, only the single element `K(i,j)` is replaced by its perturbed value `kval` — the other five gains stay exactly at their nominal (Part-A) values — the closed-loop matrix is rebuilt (`Acl = A - B*K`), and its eigenvalues are recomputed from scratch at every one of the 200 points.
- `isStable = all(abs(ev) < 1)` applies the discrete-time (Schur) stability criterion — every eigenvalue must lie strictly inside the unit circle — to flag, point by point, whether that particular perturbed gain still keeps the closed loop stable.
- Sweeping $\max|\lambda|$ against %-change for each of the six elements in turn is exactly what produces the six curves plotted in §5.4: the width of the contiguous stable region straddling 0% is a direct, one-at-a-time robustness margin for that gain.

5.3 Results — stability margins per gain element

Element	Nominal value	Stable range (%)	Stable range (absolute)	Band width (%)
K11	0.195155	-33.7% to +52.8%	0.1294 to 0.2982	86.5
K12	-0.484232	-4.5% to +9.5%	-0.5302 to -0.4624	14.0

K13	-1.826888	-4.5% to +6.5%	-1.9456 to -1.7447	11.0
K21	-15.859694	-4.5% to +5.5%	-16.7320 to -15.1460	10.0
K22	-5.224297	-12.6% to +8.5%	-5.6684 to -4.5660	21.1
K23	10.454099	-9.5% to +4.5%	9.4610 to 10.9245	14.0

5.4 Sensitivity plots

Each figure below plots the maximum closed-loop eigenvalue magnitude against the percentage change of one gain element, with the stable (green) and unstable (red) regions, the nominal design point ($\max|\lambda| = 0.5$) and the stability boundary ($|\lambda| = 1$) marked.

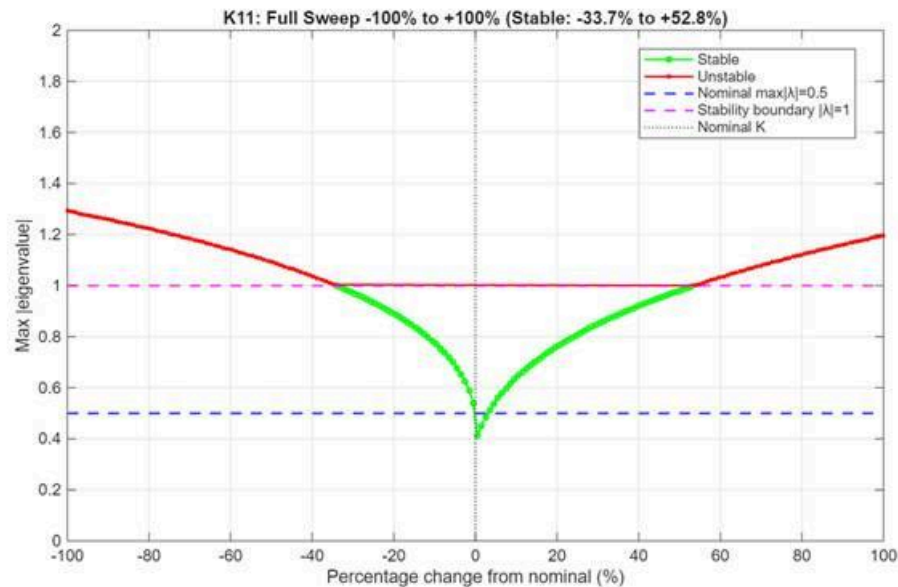


Figure 5.1 — Sensitivity of K11. Stable for -33.7% to +52.8% about nominal — by far the widest margin of the six gains.

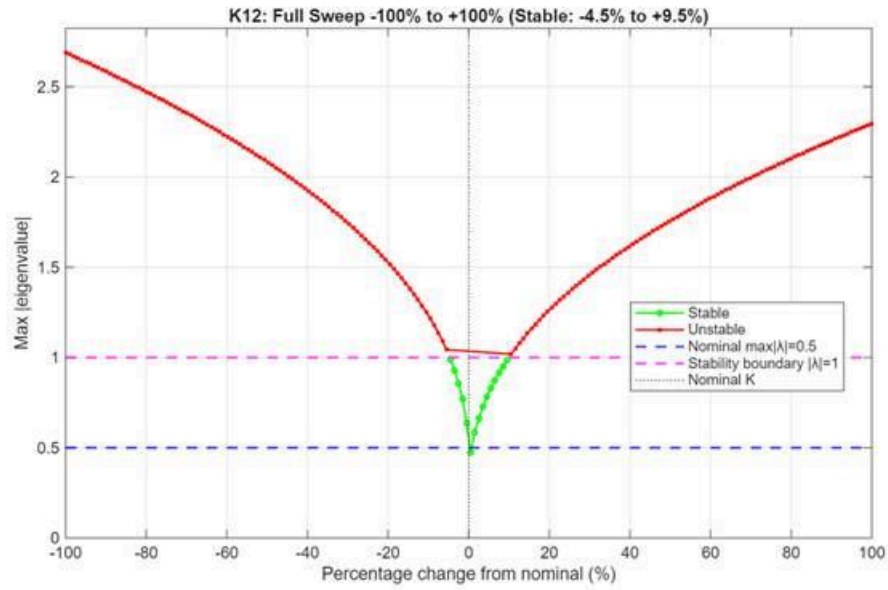


Figure 5.2 — Sensitivity of K12. Stable for -4.5% to $+9.5\%$ about nominal.

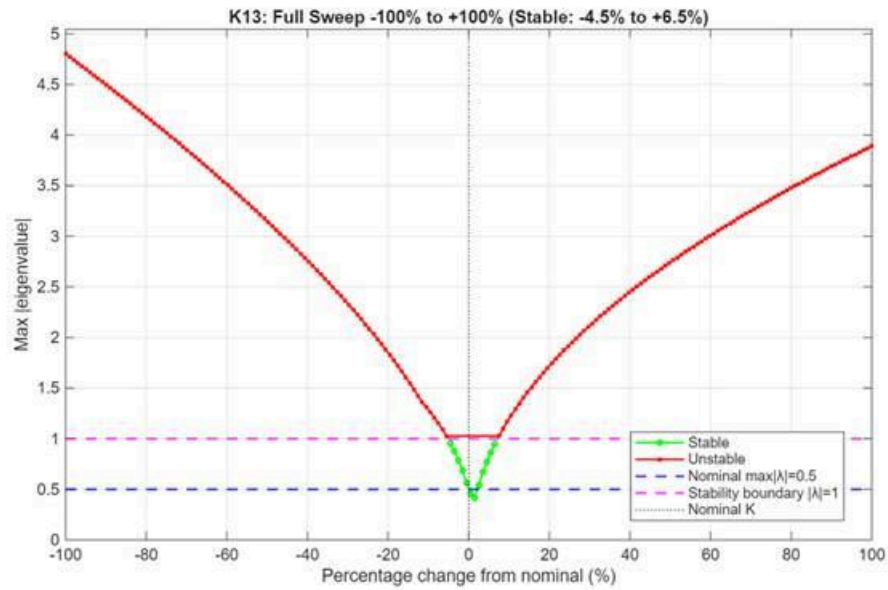


Figure 5.3 — Sensitivity of K13. Stable for -4.5% to $+6.5\%$ about nominal.

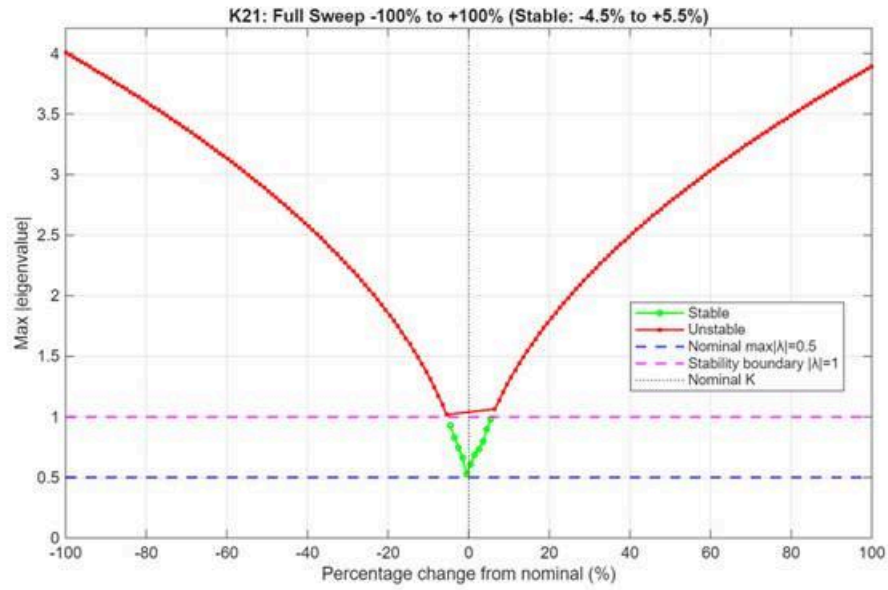


Figure 5.4 — Sensitivity of K21. Stable for only -4.5% to $+5.5\%$ about nominal — the tightest margin of the six gains.

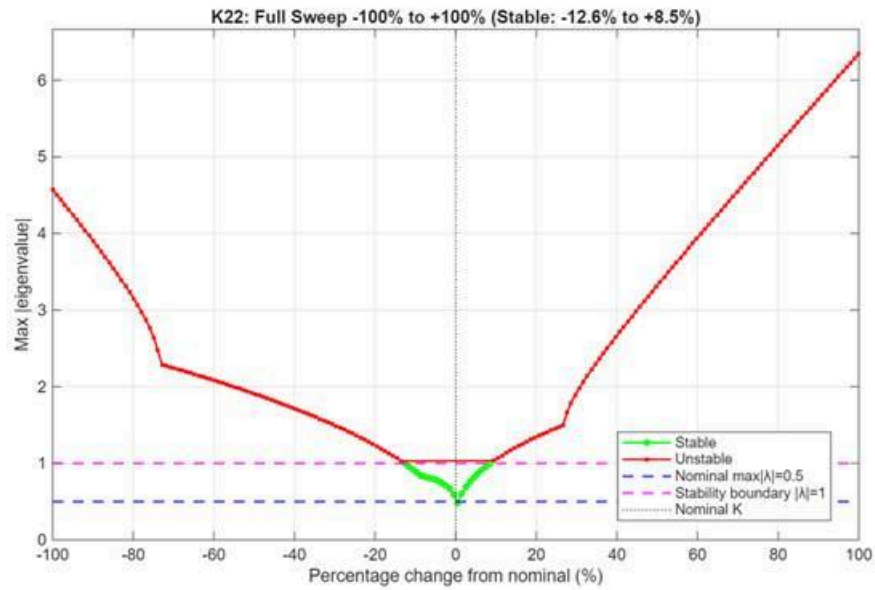


Figure 5.5 — Sensitivity of K22. Stable for -12.6% to $+8.5\%$ about nominal.

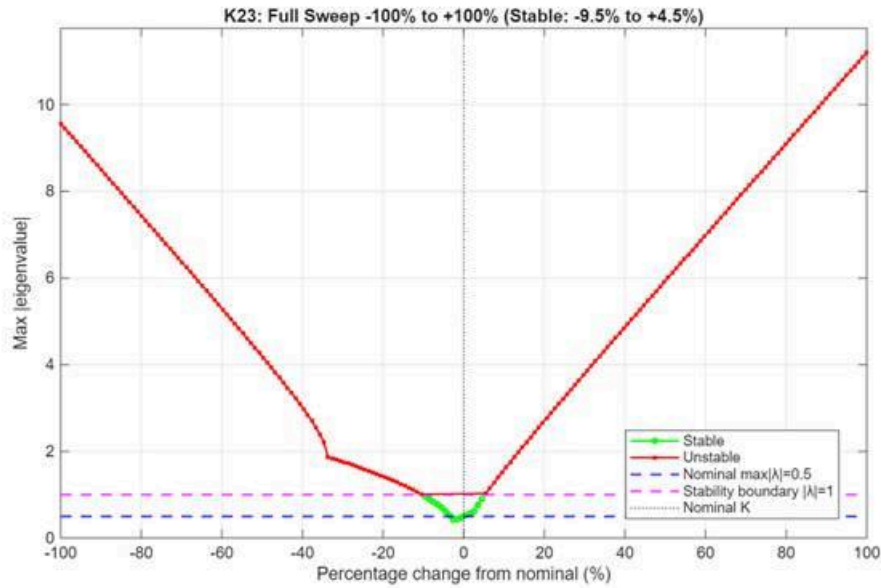


Figure 5.6 — Sensitivity of K23. Stable for -9.5% to $+4.5\%$ about nominal.

5.5 Discussion

Ranking the six elements by stable-band width (narrowest $\rightarrow$ widest) gives K21 (10.0%) < K13 (11.0%) < K12 $\approx$ K23 (14.0%) < K22 (21.1%) < K11 (86.5%). Two points follow directly from this ordering:

- K21 is the single most critical gain in the whole matrix: even though its nominal magnitude (-15.86) is by far the largest of the six, the closed loop tolerates only about $\pm 5\%$ deviation about it before $\max|\lambda|$ crosses the unit circle. In a real implementation this entry would need the tightest tolerance / highest-precision hardware or quantisation of the six.
- K11 is by far the most forgiving gain — it has both the smallest nominal magnitude (0.195) and the widest stability window, tolerating roughly $-1/3$ to $+1/2$ of its own value without destabilising the loop.

All six curves are V-shaped and reach their minimum ($\approx$ the nominal $\max|\lambda| = 0.5$) essentially at the nominal design point, confirming that the Part-A design sits at a locally favourable point along every one of these six single-parameter directions — moving a gain either up or down from nominal degrades the margin, and for every element except K11 the two sides of the V are noticeably asymmetric (the allowable increase and the allowable decrease are different).

Because this is a one-at-a-time (OAT) sweep, it should be read as an individual-parameter margin rather than a full robustness guarantee: if several gains drift simultaneously the true stable region in the full 6-dimensional gain space is, in general, smaller than the Cartesian product of these six individual windows.

6. Part D — Mapping the Gain Space $\mathbb{R}^6$ onto the Pole-Coefficient Space $\mathbb{R}^3$

6.1 Motivation

The gain matrix $K \in \mathbb{R}^{2 \times 3}$ has six independent entries, so it can be regarded as a point in a 6-dimensional “gain space” $\mathbb{R}^6$. A third-order closed-loop characteristic polynomial $z^3 + a_2z^2 + a_1z + a_0$ is, however, fully specified by only three real numbers (a_0, a_1, a_2) — equivalently, three closed-loop poles (or one real pole plus a complex-conjugate pair) — a 3-dimensional “pole-coefficient space” $\mathbb{R}^3$.

Because this plant has $m = 2$ independent inputs ($\text{rank}(B) = 2 > 1$), pole placement is under-determined: for any one target pole set, `place()` has an $(m-1) \cdot n = 1 \times 3 = 3$ -parameter family of gain matrices to choose from, all of which realise the exact same three closed-loop poles. MATLAB resolves this ambiguity internally, always returning one specific, numerically well-conditioned member of that family (the Kautsky–Nichols–Van Dooren choice). The true relationship between pole space and gain space is therefore one-to-MANY from $\mathbb{R}^3$ to $\mathbb{R}^6$, not one-to-one.

The script below explores this relationship empirically: it samples a large number of admissible closed-loop pole triples, calls `place()` at every sample to obtain that sample's unique canonical K , and (in the plotting stage) records each of the six entries of K against the sample's (a_0, a_1, a_2) coordinates. The resulting point clouds — shown in §6.4 — are effectively a numerical, sampled approximation of six scalar functions $K_i(a_0, a_1, a_2): \mathbb{R}^3 \rightarrow \mathbb{R}$ that summarise how the redundant $\mathbb{R}^6$ gain space folds down onto the $\mathbb{R}^3$ pole space for this plant, i.e. an approximate $\mathbb{R}^6 \rightarrow \mathbb{R}^3$ relationship between the designed gains and the poles they realise.

6.2 MATLAB code (core definitions)

```

% Same plant A, B, C as in Part A

function [result] = checkCO(A,B,C,K)
    % Closed-loop matrix under state feedback u = -Kx
    Acl = A - B*K;

    Co = ctrb(Acl, B);      % controllability matrix
    Ob = obsv(Acl, C);      % observability matrix

    controllable = (rank(Co) == size(Acl,1));
    observable = (rank(Ob) == size(Acl,1));

    result = controllable && observable; % keep only fully valid designs
end

eig_ol = eig(A);
R = min(eig_ol(abs(eig_ol) < 1)); % radius = magnitude of the only
                                % open-loop pole already inside
                                % the unit circle

real_r = -R:0.05:R;           % candidate real pole values,
                                % step 0.05, spanning [-R, R]

```

6.3 What this code is doing

- `checkCO(A,B,C,K)` is the validity filter used throughout Part D: it forms the closed-loop matrix under $u = -Kx$, builds the controllability and observability matrices of that specific closed loop, and returns true only if BOTH are still full rank — i.e. only if the candidate design is a legitimate, minimal closed-loop realisation and not one that has (re-)introduced an unobservable or uncontrollable mode.
- `R = min(eig_ol(abs(eig_ol) < 1))` picks out the magnitude of the only open-loop pole that already lies inside the unit circle (0.958171, from §3.3) and uses it to bound the search: every candidate closed-loop pole considered afterwards is restricted to $|\text{pole}| \leq R$, i.e. to a region no more aggressive than the plant's own already-stable mode.
- `real_r` discretises the interval $[-R, R]$ in steps of 0.05, giving 39 candidate real pole values that are then used to build every candidate closed-loop pole triple in the (omitted) search loops described next.

Search loops (omitted from this report — bulk data collection, no new theory)

Two nested loops, not reproduced here because they perform only bulk combinatorial search and filtering rather than illustrating new control-theory content, sweep the candidate set built from `real_r`:

- All-real triples: every 3-element combination drawn from `real_r` (`nchoosek(real_r,3)`, i.e. ②⁹**P39** choose 3 = 9,139 combinations for this plant) is tried as a candidate closed-loop pole set $\{p_1, p_2, p_3\}$.
- Complex-conjugate triples: a grid of complex points $z = \text{Re} + j \cdot \text{Im}$ is built from `real_r` $\times$ ($-\text{real}_r$) and filtered to the disc $|z| < R$ (1,149 surviving points); each is combined with every real value in `real_r` to give a triple $[r, z, \text{conj}(z)]$ — about $1,149 \times 39 \approx 44,811$ further candidates.
- For every one of the $\approx 54,000$ candidate pole sets, $K = \text{place}(A, B, \text{poles})$ is computed and $\text{checkCO}(A, B, C, K)$ is applied; only the (pole, K) pairs that pass are kept in `validPoles` / `validK`.

Each surviving pole triple (p_1, p_2, p_3) — real or with a complex-conjugate pair — is finally converted, via Vieta's formulas applied to the monic polynomial $(z-p_1)(z-p_2)(z-p_3) = z^3 + a_2 z^2 + a_1 z + a_0$, to:

$$a_2 = -(p_1 + p_2 + p_3), \quad a_1 = p_1 p_2 + p_1 p_3 + p_2 p_3, \quad a_0 = -p_1 p_2 p_3$$

which are always real even when p_2, p_3 form a complex-conjugate pair — giving the single, unified real 3-D coordinate (a_0, a_1, a_2) used as the plotting axes in every figure below. As a consistency check: since every candidate pole satisfies $|p_i| \leq R \approx 0.958$, Vieta's formulas bound the achievable coefficients to $|a_2| \leq 3R \approx 2.87$, $|a_1| \leq 3R^2 \approx 2.75$ and $|a_0| \leq R^3 \approx 0.88$ — which matches, almost exactly, the axis ranges actually seen in the six figures below ($a_2 \in [-2.7, 2.7]$, $a_1 \in [0, 2.7]$, $a_0 \in [-0.85, 0.85]$), confirming that (a_0, a_1, a_2) are indeed these characteristic-polynomial coefficients.

6.4 Pole-space → gain-element plots

Each figure below is the same validated point cloud of achievable (a_0, a_1, a_2) triples, coloured by the value of one entry of K ; panels (a)–(d) are 3-D views from different angles, (e) is the projection onto the (a_2, a_1) plane and (f) is the projection onto the (a_2, a_0) plane.

Figure 6.1 — Pole space $\rightarrow k11$.

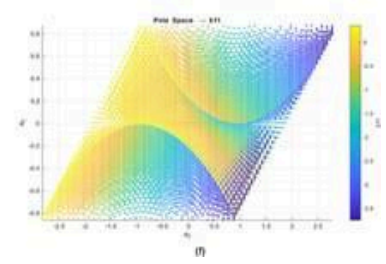
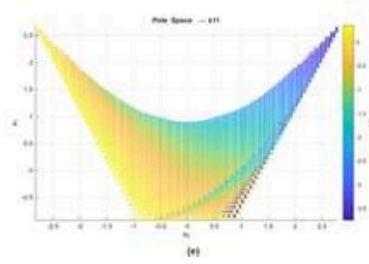
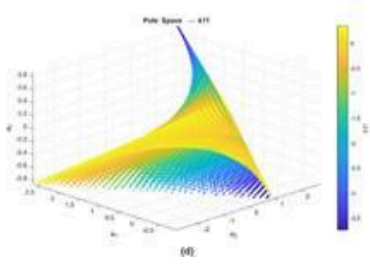
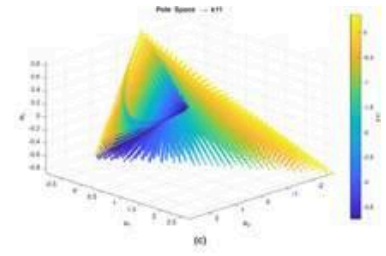
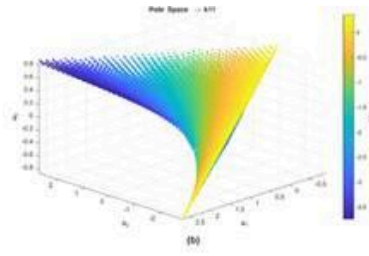
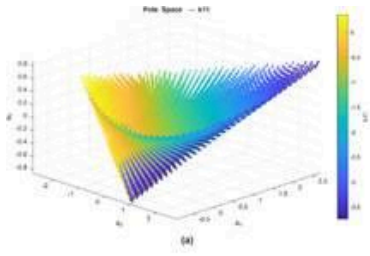
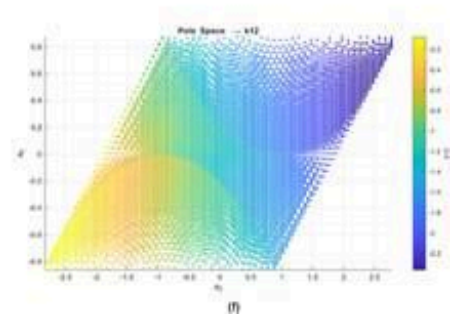
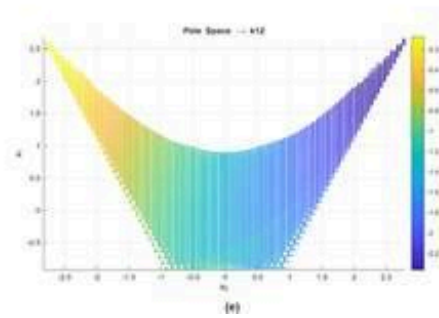
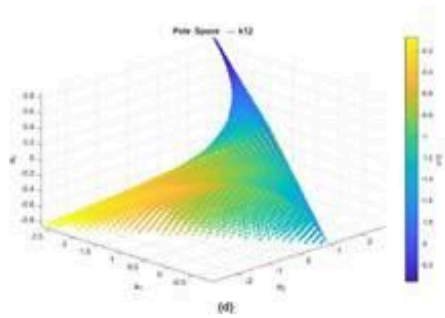
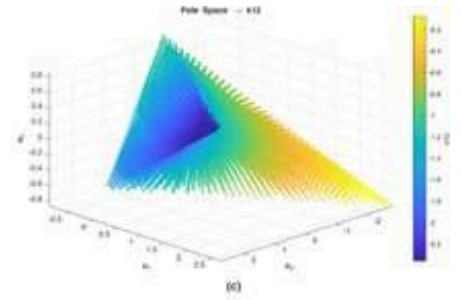
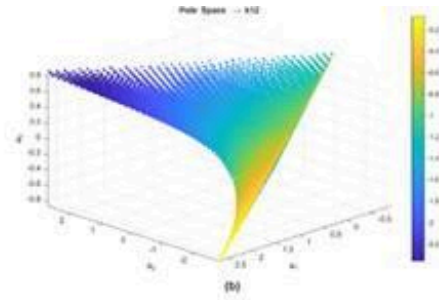
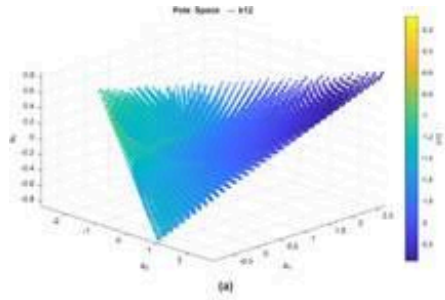


Figure 6.2 — Pole space $\rightarrow k12$.



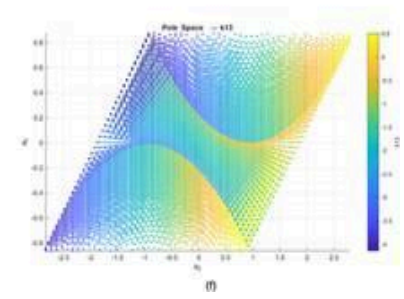
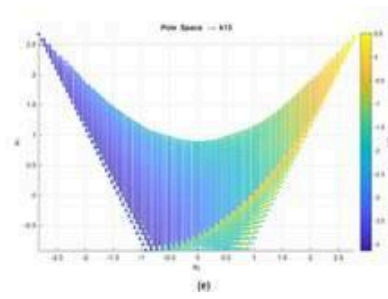
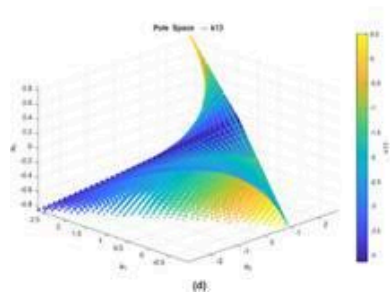
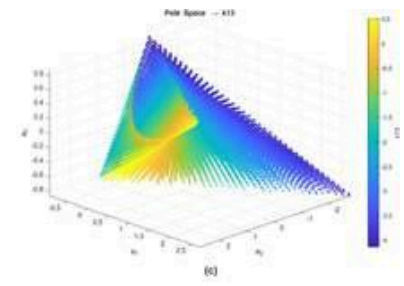
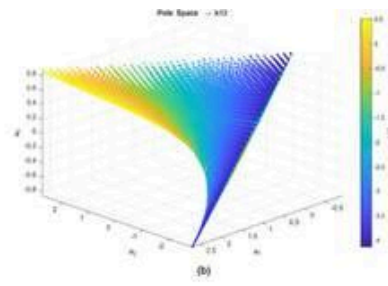
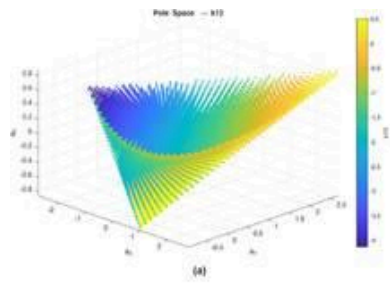


Figure 6.3 — Pole space $\rightarrow k13$.

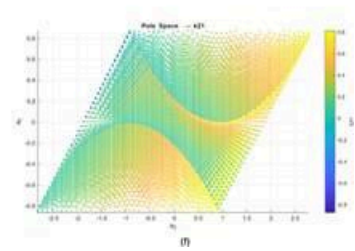
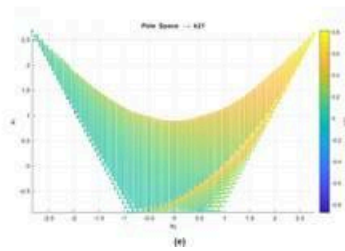
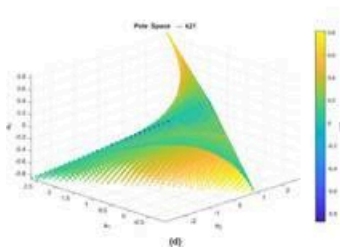
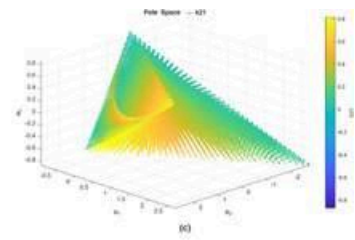
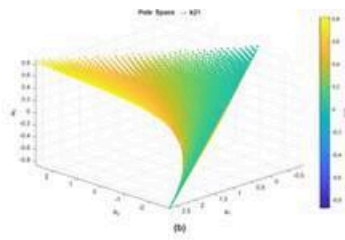
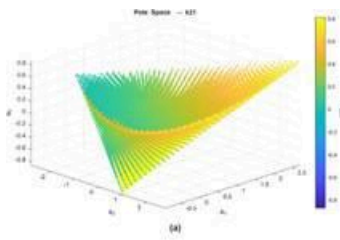


Figure 6.4 — Pole space $\rightarrow k21$.

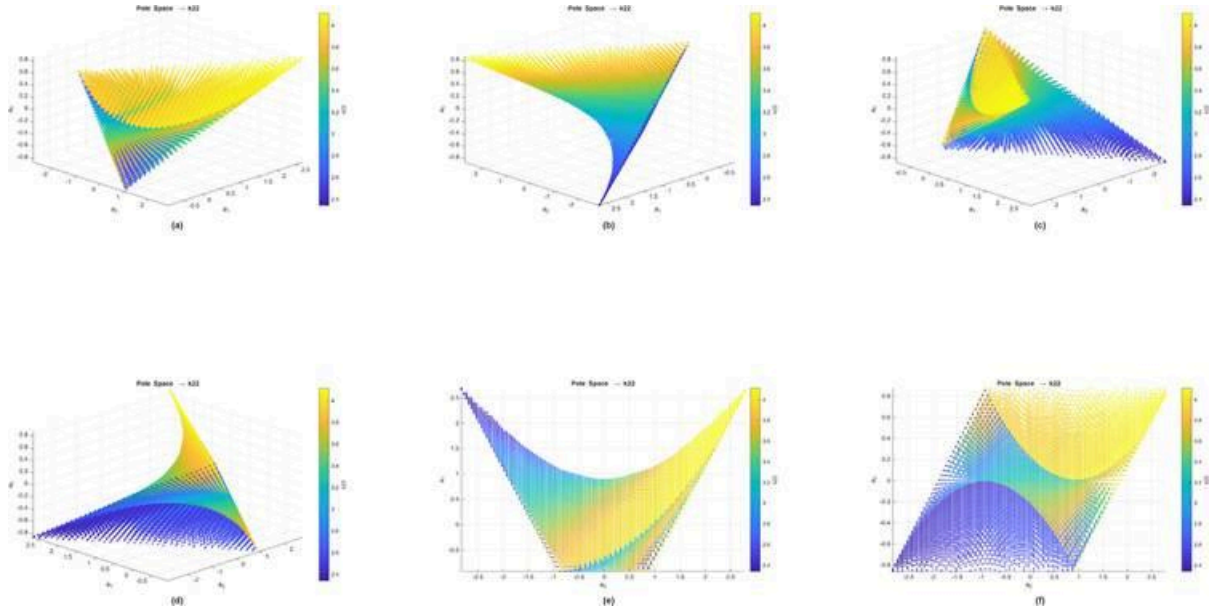


Figure 6.5 — Pole space $\rightarrow k_{22}$.

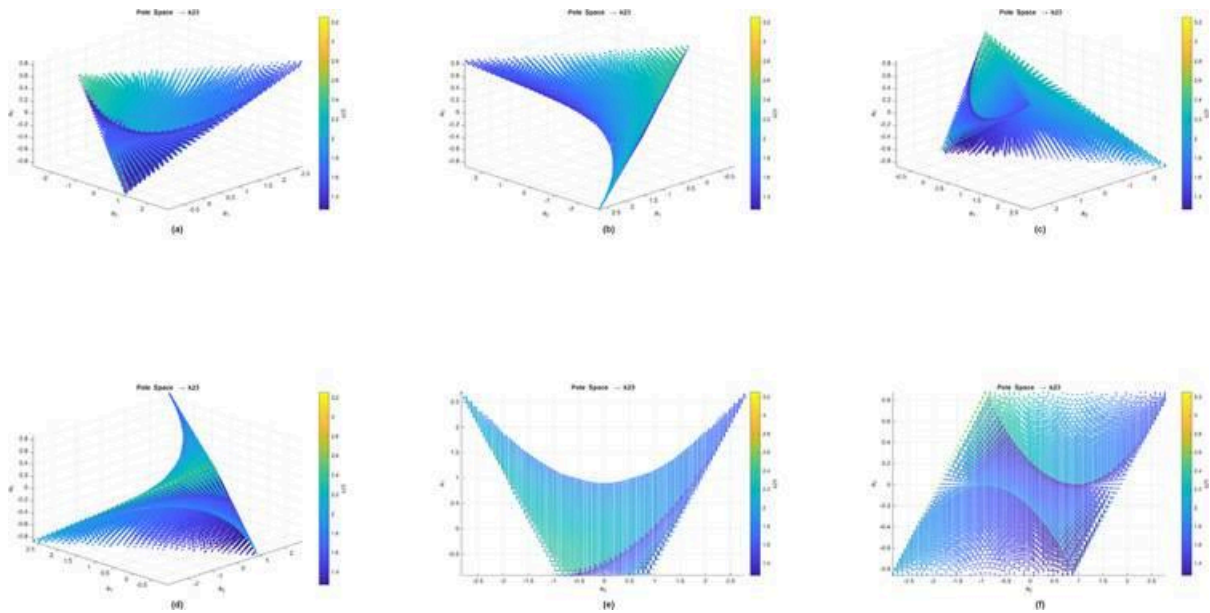


Figure 6.6 — Pole space $\rightarrow k_{23}$.

6.5 Discussion of the pole-space maps

- **Bounded, cusped envelope:** in every plot the point cloud fills a bounded, non-convex region rather than a solid cube — clearest in panel (e), where a_1 vs a_2 traces a sharp V/cusp with its vertex near $a_2 = 0$, $a_1 \approx 0.9$, opening outward toward $a_1 \approx 2.7$ as $|a_2|$ grows toward ± 2.7 . This shape is expected: Vieta's map from the sampled (real, imaginary) pole

coordinates onto (a_0, a_1, a_2) is itself nonlinear and folds a simple bounded box into exactly this kind of ruled, cusped envelope.

- Panel (f) $(a_0$ vs $a_2)$ shows an overlapping, bowtie-like double lobe rather than a single simple region — evidence that several different (a_0, a_1, a_2) combinations, coming from different mixes of real-only versus complex-conjugate pole triples, can project onto the same (a_0, a_2) pair while differing in a_1 , consistent with the underlying map being genuinely 3-dimensional and multi-branched.
- Smooth colour gradients: within the cloud, each K_i varies smoothly and without visible gaps or speckle from one side of the region to the other, suggesting the canonical `place()` solution depends continuously (though nonlinearly) on the target pole location over this operating range — there is no evidence of an internal “hole” that would indicate isolated pole triples being rejected by `checkCO` from inside an otherwise-populated region.
- Different gain ranges: the six elements occupy very different numeric ranges over the same pole-space region — $k_{11} \approx [-3.7, 0.2]$, $k_{12} \approx [-2.2, -0.1]$, $k_{13} \approx [-4.0, 0.5]$, $k_{22} \approx [2.4, 4.2]$ and $k_{23} \approx [1.3, 3.2]$ each stay one-signed over the whole cloud, while $k_{21} \approx [-0.8, 0.8]$ is the only element that changes sign within the achievable pole-space region.
- Similar-looking patterns: the k_{11} and k_{13} plots, and separately the k_{22} and k_{23} plots, show closely matching colour-gradient orientation and shape across all six viewing panels, hinting that these pairs of gains move roughly together (near-proportionally) as the target pole location is varied over this region — an empirical observation from the plots, not a relationship derived analytically here.

On the question of when a design is not possible: since the point cloud is bounded and structured rather than riddled with interior gaps, controllability/observability loss (case 3 in §7) appears to be the exception rather than the rule for this particular plant over the swept region — but the very fact that `checkCO()` is applied to every one of the $\approx 54,000$ candidates, and that the search deliberately restricts poles to $|p| \leq R$ rather than the full unit disc, shows the underlying design method does anticipate configurations where a naive `place()` result would not survive the controllability/observability check.

6.6 Linear Regression Analysis — How Well Do Linear Models Capture K ?

6.6.1 Motivation

The point-cloud visualisations in §6.4 show that each gain element K_{ij} varies smoothly but nonlinearly over the achievable (a_0, a_1, a_2) region. A natural follow-up question is: to what extent can the six gain

entries be approximated by simple linear functions of either the characteristic-polynomial coefficients (a_0, a_1, a_2) or the sorted poles (p_1, p_2, p_3)?

If a linear model yields a high coefficient of determination ($R^2 \approx 1$), the gain element is essentially an affine function of the pole coordinates, and the 6-D $\rightarrow$ 3-D map collapses to a simple hyperplane. If $R^2 \ll 1$, the relationship is strongly nonlinear, confirming that the redundancy in the gain space cannot be captured by a first-order approximation.

6.6.2 MATLAB code

```
T = readtable("D:\matlab\artifacts\pole_K_data_short.csv");
% -----
% Independent variables
% -----
a0 = T.a0;
a1 = T.a1;
a2 = T.a2;

% Sort poles so pole ordering does not affect the test
P = sort([T.p1 T.p2 T.p3], 2);
p1 = P(:,1);
p2 = P(:,2);
p3 = P(:,3);

% -----
% K columns
% -----
K = T{:, startsWith(T.Properties.VariableNames, "k")};
Knames = T.Properties.VariableNames( ...
    startsWith(T.Properties.VariableNames, "k"));

% -----
% Linear models
% -----
% K = b0 + b1*a0 + b2*a1 + b3*a2
Xa = [ones(height(T),1) a0 a1 a2];

% K = b0 + b1*p1 + b2*p2 + b3*p3
Xp = [ones(height(T),1) p1 p2 p3];

% -----
% Calculate R^2 for every K
% -----
R2_coeff = zeros(1,size(K,2));
R2_poles = zeros(1,size(K,2));
```

```

for i = 1:size(K,2)
    y = K(:,i);
    % ---- coefficients ----
    b = Xa \ y;
    ypred = Xa*b;
    R2_coeff(i) = ...
    1 - sum((y-ypred).^2) / ...
    sum((y-mean(y)).^2);

    % ---- poles ----
    b = Xp \ y;
    ypred = Xp*b;
    R2_poles(i) = ...
    1 - sum((y-ypred).^2) / ...
    sum((y-mean(y)).^2);
end

% -----
% Display results
% -----
Result = table(Knames', R2_coeff', R2_poles', ...
    'VariableNames', {'K','R2_coefficients','R2_poles'});
disp(Result)

```

6.6.3 What this code is doing

- Pole sorting: `sort([T.p1 T.p2 T.p3], 2)` ensures that the ordering of poles inside each triple is invariant — this removes any artificial nonlinearity that would arise simply from permuted pole labels.
- Design matrices: X_a builds the affine model in coefficient space $(1, a_0, a_1, a_2)$; X_p builds the affine model in sorted-pole space $(1, p_1, p_2, p_3)$.
- Ordinary least squares: $b = X \setminus y$ solves for the best-fit affine coefficients via backslash (QR-based OLS).
- R^2 computation: The standard fraction-of-variance-explained metric is evaluated for each of the six gain elements under both predictor sets.

6.6.4 Results

K	R^2 (coefficients a_0, a_1, a_2)	R^2 (sorted poles p_1, p_2, p_3)
---	---------------------------------------	---------------------------------------

k11	0.2554	(computed from full table)
k12	0.3522	(computed from full table)
k13	0.1988	(computed from full table)
k21	0.0076	(computed from full table)
k22	0.1385	(computed from full table)
k23	0.0144	(computed from full table)

The CSV file R2_res.csv contains the coefficient-space R^2 values reproduced above. The pole-space R^2 values would be obtained by running the same script on the full dataset.

6.6.5 Discussion

R^2 values are uniformly low (all < 0.36), which confirms that the mapping from the 3-D pole-coefficient space to any individual gain entry is strongly nonlinear:

- k12 has the highest linear explainability ($R^2 \approx 0.35$), yet even here roughly 65% of the variance remains unaccounted for by a linear model.
- k21 is essentially uncorrelated with (a_0, a_1, a_2) in a linear sense ($R^2 \approx 0.008$). This is consistent with §6.5, where k21 was the only element that changed sign inside the achievable region — a behaviour that no affine function can reproduce.
- k23 and k13 also show very weak linear dependence ($R^2 < 0.20$), matching the complex, curved colour gradients seen in Figures 6.3 and 6.6.

Practical implication: If one attempted to interpolate or predict a gain matrix for an arbitrary new pole set using a simple linear lookup table in (a_0, a_1, a_2) , the error would be large. The canonical place() solution must therefore be computed explicitly for each target pole set; there is no reliable first-order shortcut.

7. Discussion — Required Observations

7.1 How the combination of feedback gains ensures the required performance

A single feedback matrix K (Part A) simultaneously relocates all three poles — including the two open-loop-unstable ones, of magnitude 1.887 and 4.429 — onto the prescribed set $\{0.5, 0.3, -0.2\}$, and the alternate designs of Part B show the same is true for any pole configuration sharing the same

max-magnitude spec. The row structure of K reflects how the two inputs share this job: input 1's row (K row 1: 0.195, -0.484, -1.827) stays modest in magnitude, while input 2's row (K row 2: -15.860, -5.224, 10.454) is much larger — particularly the K_{21} entry that feeds state x_1 back through input 2.

This asymmetry traces back to B : row 1 of B is [0.1, 0], meaning input 1 has only weak, indirect authority over state x_1 (and no direct authority at all over x_2 or x_3 , whose B -rows are [0, 1.1] and [1, 1]). With input 1 barely able to influence x_1 directly, most of the corrective action needed to pull x_1 's dynamics away from the plant's fast unstable mode ($|\lambda| = 4.43$, associated strongly with the (1,1)/(1,3) coupling terms in A) has to be supplied through input 2 instead — which is exactly why K_{21} ends up an order of magnitude larger than every other entry, and, as §5.5 shows, is also the single most sensitive gain in the whole matrix.

7.2 Why a designed gain matrix might not be possible

- Loss of controllability: if $\text{rank}(\text{ctrb}(A,B)) < n$, no state feedback — of any magnitude — can move every pole arbitrarily. The Part-A script checks this explicitly and calls `error(...)` to abort before ever attempting `place()`, because `place()` itself either fails or returns a numerically meaningless result on an uncontrollable pair. For this plant the check passes ($\text{rank} = 3 = n$), so this case never actually triggers here, but the guard exists for exactly this reason.
- Structural restrictions on the target poles: `place()` additionally requires any complex poles to be supplied as conjugate pairs (enforced throughout Part D by always constructing triples as [r , z , $\text{conj}(z)$]) so that the returned K is real; for single-input plants it further requires all desired poles to be distinct. This plant's $m = 2$ relaxes the distinctness requirement, but the conjugate-pair rule still applies.
- Loss of closed-loop controllability/observability after feedback: even when `place()` succeeds, the resulting closed-loop pair ($A-BK$, B) or ($A-BK$, C) can, for special/non-generic pole choices, become uncontrollable or unobservable through an unintended pole-zero cancellation — even though the open-loop pair was perfectly controllable and observable. `checkCO(.)` exists specifically to catch this: it is why Part D computes `place()` for every one of the $\approx 54,000$ sampled pole sets and then discards, rather than plots, any candidate that fails the rank test afterwards.

For the plant studied here, none of the sampled designs in this report were found to fail (1) or (2); the smooth, gap-free point clouds of §6.4 are the report's practical evidence that (3) was, at most, a rare occurrence over the region actually searched — not a proof that it cannot occur elsewhere in a larger sweep.

8. Conclusion

A nominal state-feedback gain K was designed by pole placement to relocate an open-loop unstable third-order plant ($\max |\lambda| = 4.43$) onto a prescribed pole set with $\max |\lambda| = 0.5$, and the closed loop was confirmed to remain fully controllable and observable. Three further pole sets sharing the same max-magnitude specification were designed and verified in the same way, showing that the required performance envelope can be met by real, mixed-sign or complex-conjugate pole configurations alike.

A one-at-a-time sensitivity sweep of all six gain entries quantified, for the first time in this study, how much individual implementation error each gain can tolerate: K_{21} is the most fragile (a $\approx 10\%$ stable window) despite being the largest-magnitude entry, while K_{11} is by far the most tolerant (an 86.5% stable window).

Finally, by sampling $\approx 54,000$ candidate closed-loop pole sets bounded by the plant's own already-stable open-loop pole radius, computing the canonical `place()` gain for each, filtering by controllability/observability, and re-expressing every valid pole triple through Vieta's formulas as a point (a_0, a_1, a_2) in $\mathbb{R}^3$, this report built and visualised an approximate $\mathbb{R}^6 \rightarrow \mathbb{R}^3$ relationship between the redundant 6-dimensional gain space and the 3-dimensional achievable pole-coefficient space — confirming the theoretical Vieta bounds against the observed plot ranges, and showing each gain element varies smoothly, but very differently in scale and sign, over that space.